Diving Deeper – Circular Queues and Deques

Now that we've got a solid understanding of Simple Queues, it's time to take the plunge and dive deeper into the world of Queues. In this module, we'll explore two other types of Queues – Circular Queues and Deques (Double Ended Queues).

Understanding Circular Queues

The Concept of a Circular Queue

In a Circular Queue, the last element points back to the first element making a circular link. We can visualize it as a circle where we remove elements from one end and add elements at the other end, and it goes on in a cycle.

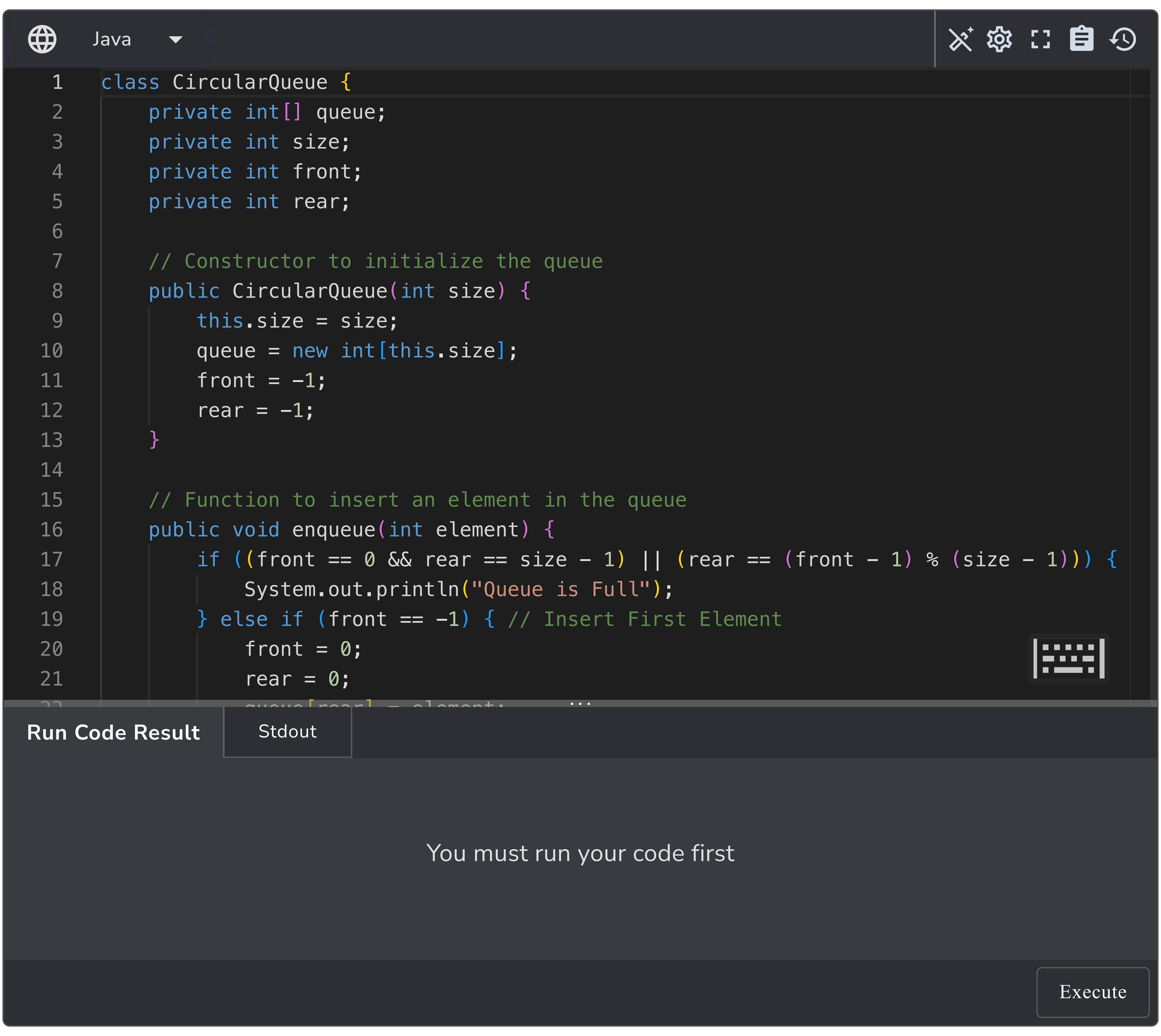
Operations on a Circular Queue

We will use an array to implement circular queue.

The operations on a Circular Queue are similar to those of a Simple Queue – Enqueue, Dequeue, Peek, and IsEmpty. But there's a twist in the way we handle these operations due to the circular nature of the Queue.

When we Enqueue (add an element), we add it at the rear and increment the rear pointer. But if we reach the end of our array while enqueueing, instead of declaring an Overflow, we wrap around and continue from the front of the array, as long as there is space.

Similarly, when we Dequeue (remove an element), we remove it from the front and increment the front pointer. But if we reach the end of the array while dequeuing, we wrap around and continue from the start of the array.



```java
class CircularQueue {
    private int[] queue;
    private int size;
    private int front;
    private int rear;

    // Constructor to initialize the queue
    public CircularQueue(int size) {
        this.size = size;
        queue = new int[this.size];
        front = -1;
        rear = -1;
    }

    // Function to insert an element in the queue
    public void enqueue(int element) {
        if ((front == 0 && rear == size - 1) || (rear == (front - 1) % (size - 1))) {
            System.out.println("Queue is Full");
        } else if (front == -1) { // Insert First Element
            front = 0;
            rear = 0;
```

You must run your code first

Common Mistakes and How to Avoid Them

While working with Queues, it's easy to make a few common mistakes. The first mistake is forgetting to check for Queue Overflow and Underflow conditions. Always remember to check if the Queue is full before enqueueing and if it's empty before dequeuing. This will save you from unexpected errors.

Another common mistake is mixing up the front and rear pointers. Keep in mind that we always add items to the rear and remove them from the front.

Introduction to Deques (Double-Ended Queues)

The Flexibility of Deques

The name Deque is short for Double Ended Queue, and as the name suggests, it's a Queue where we can add or remove elements from both ends.

Think about a library line where you can borrow books from one end and return books at the other end. The Deque is flexible and allows us to work from both ends!

Operations on Deques

The operations on Deques are a bit different from our regular Queues. We have four main operations in Deques:

- **InsertFront**: add an element at the front of the Deque.
- **InsertRear**: add an element at the rear of the Deque.
- **DeleteFront**: remove an element from the front of the Deque.
- **DeleteRear**: remove an element from the rear of the Deque.

This flexibility of Deques can be really useful in certain scenarios and can give you an extra edge when tackling complex problems.